Análisis y Reflexión Taller 2 IA

Integrantes del equipo:

- Carolina Ceballos Gómez - 202510267
- Juan Felipe Leon - 202510397
- Miguel Aguilar - 202411646
- Andrés Sanabria - 202411507

a) ANÁLISIS CSP

1. Backtracking_search:

- Complejidad en espacio: **O(n)**, n = número de variables
- Complejidad en tiempo: **O(d^n · c)**, n = número de variables, d = tamaño máximo del dominio, c = número de restricciones.
- ¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.
  La búsqueda exhaustiva simple hace todas las asignaciones posibles y para cada layout se puede calcular haciendo d^n, donde d= #drones y n = #entregas

Tabla 1. Asignaciones en backtracking

Layout	**Búsqueda Exhaustiva Simple**	**Asignaciones**	**Reducción (asignaciones que se evitaron)**
battery_edge	1	1	0
big_fleet	4^5= 1024	7	1017
corner_challenge	2^2=4	2	2
weight_distribution	3^3=9	3	6
tight_overload	2^4=16	4	12
twin_bases	2^2=4	3	1

Este algoritmo tiene complejidad en tiempo en el peor caso O(d^n · c), pues podría intentar todas las combinaciones de asignaciones, lo que lo haría no muy eficiente. Sin embargo, como este también explora considerando las restricciones, esto lo diferencia de la búsqueda exhaustiva simple, que es explorar todas las combinaciones a la fuerza. Por esto mismo se observa una reducción notoria en las asignaciones (ver tabla 1). Por ejemplo, en el layout tight_overload, el algoritmo evita hacer 12 asignaciones (comarandola con una búsqueda exhaustiva). Entonces gracias a esto se puede concluir que la verificación de consistencia

mientras se busca la solución reduce considerablemente el espacio de búsqueda, aunque el factor de reducción depende del layout, de la cantidad de drones y de la cantidad de entregas.

2. **Backtracking_fc:**
   - Complejidad en espacio: **O(n · d)**, n = número de variables, d = tamaño máximo del dominio
   - Complejidad en tiempo: **O(d^n · c · d)**, n = número de variables, d = tamaño máximo del dominio, c = número de restricciones.
   - ¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.
     La búsqueda exhaustiva simple hace todas las asignaciones posibles y para cada layout se puede calcular haciendo d^n, donde d= #drones y n = #entregas

Tabla 2. Asignaciones en backtracking con forward checking

Layout	**Búsqueda Exhaustiva Simple**	**Asignaciones**	**Reducción (asignaciones que se evitaron)**
battery_edge	1	1	0
big_fleet	4^5= 1024	7	1017
corner_challenge	2^2=4	2	2
cross_mission	2^2=4	2	2
heavy_cargo	2^3=8	3	5
weight_distribution	3^3=9	3	6
tight_overload	2^4=16	4	12
twin_bases	2^2=4	3	1

Este algoritmo tiene complejidad en tiempo en el peor caso O(d^n · c · d), pues además de explorar como lo hace backtracking también va revisando y reduciendo los dominios de los vecinos (y por esto tenemos a d en la complejidad). Pero esto también ayuda a identificar fallos o inconsistencias antes de hacer una asignación. Gracias a esto también se reduce el espacio de búsqueda y también se puede evidenciar en la tabla 2 con ejemplos de nuestros experimentos. Similar a backtracking, la reducción depende del layout, de la cantidad de drones y de la cantidad de entregas.

3. **<u>Backtracking_ac3:</u>**
   - <u>Complejidad en espacio:</u> **O(n · d + c)**, donde n = número de variables, d = tamaño máximo del dominio y c = número de restricciones.
   - <u>Complejidad en tiempo:</u> **O(c · d^n · d^3)**, n = número de variables, d = tamaño máximo del dominio
   - <u>¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.</u>
     La búsqueda exhaustiva simple hace todas las asignaciones posibles y para cada layout se puede calcular haciendo d^n, donde d= #drones y n = #entregas

Layout	**Búsqueda Exhaustiva Simple**	**Asignaciones**	**Reducción (asignaciones que se evitaron)**
big_fleet	4^5= 1024	7	1017
heavy_cargo	2^3=8	3	5
weight_distribution	3^3=9	3	6
tight_overload	2^4=16	4	12
twin_bases	2^2=4	3	1

Este algoritmo tiene complejidad en espacio O(n · d+c), ya que guarda los dominios de las variables y las restricciones entre ellas. En tiempo, su complejidad en el peor caso es O(c · d^n · d^3), porque además del backtracking, revisa las restricciones entre variables y elimina valores inconsistentes de los dominios. Esto ayuda a evitar explorar muchas combinaciones inválidas, entonces reduce bastante el espacio de búsqueda, aunque la reducción depende del layout, del número de drones y del número de entregas.

<u>Nota:</u> que los tres algoritmos tengan el mismo número de asignaciones en algunos layouts puede deberse a que todos recorren el problema en el mismo orden de variables y que aunque en algoritmos como FC o AC-3 los dominios sí se van reduciendo, tal vez no cambian mucho las decisiones que se toman.

4. **<u>Backtracking_mrv_lcv:</u>**
   - <u>Complejidad en espacio (Notación O):</u> **O(n · d+c)**, donde n = número de variables, d = tamaño máximo del dominio y c = número de restricciones

   - Complejidad en tiempo (Notación O): **O(d^n)**, donde n = número de variables y d = tamaño máximo del dominio

   - <u>¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.</u>

Layout	Búsqueda Exhaustiva	Asignaciones	Reducción
big_fleet	1024	7	1017
heavy_cargo	8	3	5
weight_distribution	27	3	24
tight_overload	16	4	12
twin_bases	4	2	2
battery_edge	27	1	26
corner_challenge	27	2	25
detour_required	27	1	26
maze_csp	81	2	79

Se reduce bastante comparándolo con una búsqueda exhaustiva simple, dado que el algoritmo de backtracking con heurísticas MRV y LCV utiliza la información de las restricciones para evitar explorar combinaciones que no pueden conducir a una solución válida. Mientras una búsqueda exhaustiva evaluaría todas las posibles asignaciones d^n, el backtracking detecta inconsistencias en asignaciones parciales y poda esas ramas antes de seguir profundizando. Por ejemplo, en el layout big_fleet la búsqueda exhaustiva implicaría evaluar 1024 combinaciones, pero el algoritmo solo intentó 7 asignaciones, evitando 1017 posibles estados. De forma similar, en weight_distribution se pasarían de 27 combinaciones posibles a solo 3 intentos, evitando 24, y en maze_csp de 81 combinaciones a 2 intentos, reduciendo 79 evaluaciones. Estos resultados muestran cuantitativamente cómo el uso de heurísticas y verificación de restricciones disminuye drásticamente el número de estados explorados.

b) ANÁLISIS BÚSQUEDA ADVERSARIA

- <u>Complejidad en tiempo:</u> **O(b^d)**, b = factor de ramificación, d = función de profundidad
- <u>Complejidad en espacio:</u> **O(b · d)**, b = factor de ramificación, d = función de profundidad
- <u>¿Por qué Minimax puede ser pesimista en entornos con cazadores no óptimos?</u>
  Puede ser pesimista en entornos con cazadores no óptimos pues él estará jugando asumiendo que sus oponentes siempre elegirán la mejor acción para minimizar la utilidad del dron (o sea que son óptimos siempre), entonces el agente puede "complicarse" en la toma de decisiones cuando puede que su contrincante solo esté tomando acciones al azar o con heurísticas que no garanticen realmente las mejores decisiones para MIN. Entonces el agente podría estar razonando o tomando decisiones que sean menos costosas que tal vez no sean las más óptimas, pero que igual lo lleven a maximizar las entregas. Es como si todo el tiempo estuviera jugando a protegerse de los cazadores en vez de estar entregando los paquetes cuando los cazadores realmente no lo están cazando o están haciendo un muy mal trabajo en ello. Este pensamiento o razonamiento pesimista sucede porque él espera que los adversarios sean óptimos y perfectamente racionales, pero esto no siempre ocurre.
- <u>Describa y justifique su función de evaluación: ¿qué características del estado usa, qué pesos asignó y por qué?</u>
  Nuestra función de estado es como una guía para que el agente tome una buena decisión, o la decisión más favorable, pues estima si el estado es deseable o favorable asignando puntajes según el progreso de la misión (hacer entregas) o el peligro que generan los cazadores. Primero lo que hace es asignar valores extremos para que el algoritmo siempre priorice estos resultados (el positivo), entonces tenemos +1000 que solo se asigna si el dron gana (misión cumplida y se entregan todos los suministros) o -1000 si ese estado lo lleva a perder (que un cazador lo capture).

  Luego para los estados que no son extremos, nuestra función considera el punteaje actual del juego, es decir, el proceso acumulado, pero también utiliza la distancia BFS a la entrega más cercana y si el dron se acerca entonces se premia sumándole 10/(distancia + 1), ya que esto hace que se prioricen los estados que lo acercan a una entrega. Pero como también debe garantizar que los cazadores no lo capturen entonces la función se encarga de penalizar al dron si está muy cerca de un cazador (si está a una distancia >=2), la penalización tiene un peso de

-100/(distancia +0.1), y gracias a esta penalización el dron intentará evitar estas zonas, dejándolo fuera del peligro de ser atrapado.

Ya por último, para que la función no se quede siempre con la idea de evitar zonas peligrosas (es decir siempre evitar las penalizaciones y tener un razonamiento "defensivo" o "conservador"), también se penaliza según las entregas que hayan pendiente, para que se pueda finalizar la misión lo más rapido posible, en este caso se da una penalización de -20, que no afecta a gran escala como la de evitar cazadores (pues esta asegura la supervivencia) pero si mantiene al dron siempre en busca de cumplir sus objetivos 8es más como una penalización guía).
Todos los pesos que se asignan toman en cuenta que el dron debe avanzar a sus objetivos pero que debe sobrevivir (no encontrándose con cazadores), y tratamos de buscar un balance con penalizaciones y premiaciones para garantizar la eficiencia y también la seguridad.

- <u>Comparación de las tasas de victoria de los agentes en diversos layouts.</u>

Mapa	**Agentes (tasa de victoria)**			
	Minimax	**Expectimax**		**AlphaBeta**
		Probabilidad	**Tasa (sobre n=20)**	
Arena	0%	0.5	11/20 = 55%	0%
		1.0	20/20 = 100%	
Desperate run	100%	0.5	20/20 = 100%	100%
		1.0	20/20 = 100%	
Hunter swarm	0%	0.5	8/20 = 40%	0%
		1.0	17/20 = 85%	
Jungle chase	100%	0.5	13/20 = 65%	100%
		1.0	20/20 = 100%	
Mountain overfly	0%	0.5	17/20 = 85%	0%
		1.0	20/20 = 100%	
		0.5	20/20 = 100%	

Hunter cage	100%	1.0	20/20 = 100%	100%
Storm tunnel	0%	0.5	20/20 = 100%	0%
		1.0	20/20 = 100%	
Terrain trap	100%	0.5	20/20 = 100%	100%
		1.0	20/20 = 100%	
Wide open	0%	0.5	20/20 = 100%	0%
		1.0	20/20 = 100%	

- <u>¿Cuántos nodos adicionales poda Alfa-Beta en los mejores y peores casos?</u>
  En nuestro código de Alpha-Beta, la poda ocurre cuando se cumple la condición en MAX de que *var* > *beta* o en MIN cuando v*ar* < *alpha*. En el peor caso donde se exploren los nodos en un orden no favorable, es decir, que los mejores movimientos o los que serán elegidos se exploren al final, entonces esas condiciones nunca se van a cumplir, se tendrá que explorar todas las ramas y no va a haber ninguna poda. En este peor caso, Alpha-Beta se comporta como MINIMAX y no se poda ningún nodo. El mejor caso, es que Alpha-Beta solo tenga que explorar un subárbol y una rama del siguiente y ya con esa información, determine que no sea necesario explorar más subárboles ni ramas, al tener los valores de *alpha* y *beta* que necesita. (donde *alpha* >= *beta*)

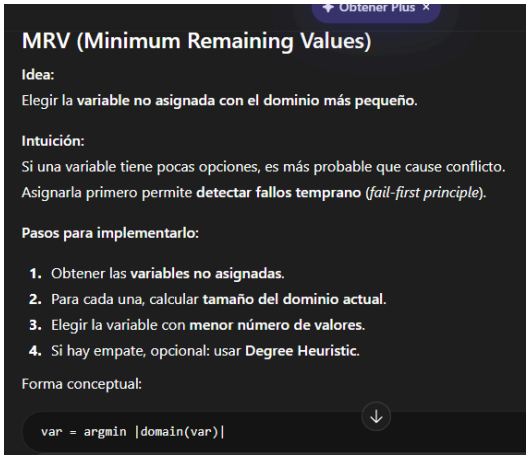
c) REFLEXIÓN

Como equipo comenzamos el desarrollo de este taller de forma autónoma sin ayudas externas, pero nos dimos cuenta que teníamos algunos vacíos conceptuales o dudas que estaban impidiendo el desarrollo del taller, así que acudimos a la IA al inicio para resolver dudas de conceptos como Consistencia de Arco, heurísticas como MRV o LCV, pues si de por sí no teníamos muy claro estos conceptos o el proceso de estos, no podiamos comenzar a implementarlos. Creemos que este apoyo fue fundamental pues nos dejó con muy buenas bases, ya que la IA nos brindaba pistas o los pasos de los algoritmos sin darnos explícitamente el código. A partir de nuestro código ya hecho, volvimos a utilizar la IA cuando nos bloqueamos o nos salía un error al probarlos, lo clave aquí fue que no le pedíamos que nos corrigiera el código y ya si no que nos diera una lista de posibles fallas de esa manera nos dabamos cuenta de los errores pero nosotros mismos los corregimos. Y a la misma vez pudimos optimizar o simplificar nuestro código para evitar redundancias o malas prácticas.

Respecto a la construcción del prompt, creemos que fue fácil hacer uno adecuado donde primero indicaremos cómo quería que nos respondiera (si de una manera técnica o más sencilla), la

inquietud o falla que teníamos, y le decíamos que sin darnos la respuesta nos guiara para llegar a ella nosotros mismos, o en algunos casos que nos diera algunas alternativas para mejorar el código. Consideramos que las respuestas que nos daba sí eran fáciles de entender (en especial porque en el prompt le pedíamos que fuera en un lenguaje sencillo sin dejar de ser técnico), y gracias a esto ya todas las dudas que le hacíamos como seguimiento si seríamos capaces de explicarlas en una sustentación. Lo más útil de la IA es su versatilidad para ayudarnos tanto en temas conceptuales como de desarrollo o incluso para verificar que las complejidades que analizamos en nuestros algoritmos sean coherentes. Finalmente mostraremos los prompts utilizados y las respuestas dadas por la IA:

Prompt para Chat-GPT4: Actúa como profesor del curso de Inteligencia Artificial, necesito que me expliques en CSP el concepto de MRV y LCV, y cómo podría programar heurísticas de esto en python, no me des el código solo dame los pasos o como pistas para yo desarrollarlo.



MRV (Minimum Remaining Values)

Idea:
Elegir la **variable no asignada con el dominio más pequeño.**

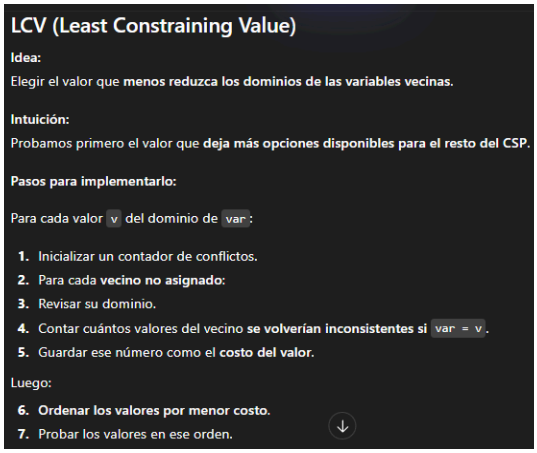
Intuición:
Si una variable tiene pocas opciones, es más probable que cause conflicto.
Asignarla primero permite **detectar fallos temprano** (*fail-first principle*).

Pasos para implementarlo:

1. Obtener las **variables no asignadas.**
2. Para cada una, calcular **tamaño del dominio actual.**
3. Elegir la variable con **menor número de valores.**
4. Si hay empate, opcional: usar **Degree Heuristic.**

Forma conceptual:

```
var = argmin |domain(var)|
```



LCV (Least Constraining Value)

Idea:
Elegir el valor que **menos reduzca los dominios de las variables vecinas.**

Intuición:
Probamos primero el valor que **deja más opciones disponibles para el resto del CSP.**

Pasos para implementarlo:

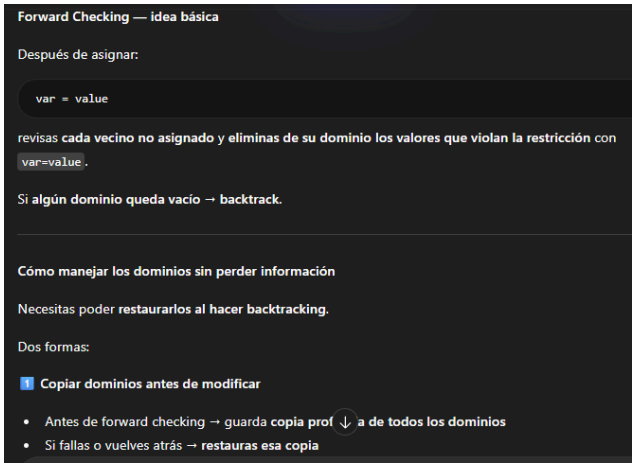
Para cada valor `v` del dominio de `var`:

1. Inicializar un contador de conflictos.
2. Para cada **vecino no asignado:**
3. Revisar su dominio.
4. Contar cuántos valores del vecino **se volverían inconsistentes si** `var = v`.
5. Guardar ese número como el **costo del valor.**

Luego:

6. **Ordenar los valores por menor costo.**
7. Probar los valores en ese orden.

Prompt para Chat-GPT4: no entiendo como debo hacer la copia del dominio y para realizar forward checking en csp, y como es que puedo ir reduciendo cada uno de los dominios, explicame con tips o pistas pero no me des codigo aun



Forward Checking — idea básica

Después de asignar:

```
var = value
```

revisas **cada vecino no asignado y eliminas de su dominio los valores que violan la restricción** con `var=value`.

Si **algún dominio queda vacío → backtrack.**

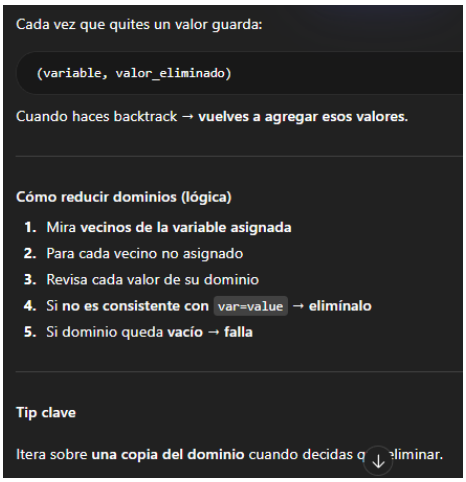
Cómo manejar los dominios sin perder información

Necesitas poder **restaurarlos al hacer backtracking.**

Dos formas:

1 **Copiar dominios antes de modificar**

- Antes de forward checking → guarda **copia prof a de todos los dominios**
- Si fallas o vuelves atrás → **restauras esa copia**



Cada vez que quites un valor guarda:

```
(variable, valor_eliminado)
```

Cuando haces backtrack → **vuelves a agregar esos valores.**

Cómo reducir dominios (lógica)

1. Mira **vecinos de la variable asignada**
2. Para cada vecino no asignado
3. Revisa cada valor de su dominio
4. Si **no es consistente con** `var=value` → **elimínalo**
5. Si dominio queda **vacío → falla**

Tip clave

Itera sobre **una copia del dominio** cuando decidas q liminar.

En Copilot Visual Studio Code:

en este caso como se acceden a las restricciones?

Para acceder a las restricciones en tu implementación de AC3, enfócate en cómo están estructuradas en el código de tu CSP (probablemente en una clase o módulo):

- **Revisa la estructura del CSP**: Busca en el código (como en `csp.py` o `problems_csp.py`) cómo se almacenan las restricciones. Comúnmente, se usan diccionarios donde las claves son pares de variables (tuplas) y los valores son las funciones o listas que definen la restricción.
- **Itera sobre las restricciones**: Para obtener los arcos, itera sobre este diccionario o lista de restricciones. Cada entrada te dará un par de variables conectadas, que puedes usar para formar los arcos (Xi, Xj) y (Xj, Xi).
- **Filtra por tipo**: Asegúrate de solo considerar restricciones binarias (las que involucran exactamente dos variables). Si hay otros tipos, ignóralos para AC3.
- **Acceso directo**: Si las restricciones están en un atributo de la clase CSP (ej. `self.constraints`), accede a él directamente en tu función AC3.

Revisa el archivo actual (`problems_csp.py`) para ver ejemplos de cómo se definen problemas específicos y cómo se pasan las restricciones al CSP. Si es un diccionario, usa métodos para obtener las claves (los pares). Si necesitas pistas sobre un formato específico, dime más detalles del código.